

```

795         center_lat + dem.dy / 2.0,
796     )
797     # Zero-area boundary touches do not occupy a nearest-neighbour cell.
798     if _polygon_area(clipped) <= 1e-18:
799         continue
800     local_vertices = [_local_xy_m(lon, lat, lon_f, lat_f) for lon, lat in clipped]
801     altitude_lb = min(plane[0] * x + plane[1] * y + plane[2] for x, y in local_vertices)
802     clearances.append(altitude_lb - float(dem.values[row, col]))
803     if clearances:
804         min_clearance_lb = min(clearances)
805         los_certified = bool(min_clearance_lb > 0.0)
806 elif candidate_rows.size and static_plan:
807     # Vertical transport motion: the plan ray is fixed. The lower of the two
808     # endpoint altitudes gives the exact worst-case line altitude for all times.
809     moving_alt = min(moving0[2], moving1[2])
810     slope = moving_alt - alt_f
811     clearances = []
812     for row, col in zip(candidate_rows.tolist(), candidate_cols.tolist()):
813         center_lon = dem.lon0 + col * dem.dx
814         center_lat = dem.lat0 - row * dem.dy
815         interval = _segment_rect_parameter_interval(
816             lon_f,
817             lat_f,
818             moving0[0],
819             moving0[1],
820             center_lon - dem.dx / 2.0,
821             center_lon + dem.dx / 2.0,
822             center_lat - dem.dy / 2.0,
823             center_lat + dem.dy / 2.0,
824         )
825         if interval is None or interval[1] - interval[0] <= 1e-12:
826             continue
827         ray_parameter = interval[0] if slope >= 0.0 else interval[1]
828         altitude_lb = alt_f + ray_parameter * slope
829         clearances.append(altitude_lb - float(dem.values[row, col]))
830     if clearances:
831         min_clearance_lb = min(clearances)
832         los_certified = bool(min_clearance_lb > 0.0)
833 elif candidate_rows.size:
834     # Rare collinear plan motion. Retain the earlier conservative envelope;
835     # failure merely triggers subdivision or the alternative relay path.
836     r_lo = np.clip(candidate_r - r_expand, 0.0, 1.0)
837     r_hi = np.clip(candidate_r + r_expand, 0.0, 1.0)
838     slopes = np.asarray([moving0[2] - alt_f, moving1[2] - alt_f], dtype=float)
839     altitude_candidates = np.stack(
840         [alt_f + r_lo * slopes[0], alt_f + r_hi * slopes[0],
841          alt_f + r_lo * slopes[1], alt_f + r_hi * slopes[1]],
842         axis=0,
843     )
844     altitude_lb = np.min(altitude_candidates, axis=0)
845     clearance_lb = altitude_lb - dem.values[candidate_rows, candidate_cols]
846     min_clearance_lb = float(np.min(clearance_lb))
847     los_certified = bool(np.all(clearance_lb > 0.0))
848
849 def distance_3d_km(point: tuple[float, float, float]) -> float:
850     h = horizontal_distance_m(point[0], point[1], lon_f, lat_f)
851     return math.hypot(h, point[2] - alt_f) / 1000.0
852
853 # Norm of an affine vector is convex, so its maximum over a closed interval is at
854 # an endpoint. This makes the distance and FSPL upper bound exact for the interval.
855 max_distance_km = max(distance_3d_km(moving0), distance_3d_km(moving1))
856 loss_upper = fspl_db(freq_mhz, max_distance_km)
857 if not los_certified:
858     loss_upper += obstruction_db
859 margin_lower = threshold_db - loss_upper
860 return {
861     "certified_available": bool(margin_lower >= -1e-12),
862     "los_certified": los_certified,
863     "loss_upper_db": loss_upper,
864     "margin_lower_db": margin_lower,
865     "min_clearance_lower_m": min_clearance_lb,
866     "candidate_cells": int(candidate_rows.size),
867     "endpoint_shift_m": endpoint_shift,
868     "max_distance_km": max_distance_km,
869 }

```